Chapter 2 worked solutions

DS02-01 • CSR and transpose

CSR is `row_ptr=[0,1,3,4]`, `col_idx=[1,0,3,1]`, and `values=[5,-2,7,4]`. Original column counts are `[1,2,0,1]`, with starts `[0,1,3,3]`. In original entry order, output positions are 1,0,3,2. The transposed COO is `(0,1,-2)`,`(1,0,5)`,`(1,2,4)`,`(3,1,7)`, with dimensions 4×3. Its column counts `[1,2,1]` and starts `[0,1,3]` transpose it back to the original dimensions and sorted triples. Check dimensions, coordinates, and values, not just nnz.

DS02-02 • Four-dimensional addresses and capacity

The element offset is $(((i_0D_1+i_1)D_2+i_2)D_3+i_3)$, or $i_0D_1D_2D_3+i_1D_2D_3+i_2D_3+i_3$. Multiply by w for the byte offset. Require $0 \le i_j < D_j$. Both the dimension product and its product with w must fit the allocation API's size type and container or platform limits. Valid indices alone do not exclude intermediate multiplication overflow. Check bounds before multiplying.

DS02-03 • KMP trace

The prefix function is $\pi = [0, 1, 2, 3, 4, 0]$. Here q is the number of matched pattern symbols. Text indices start at zero. Each row processes the same current symbol until it matches or q reaches zero; the text index never moves backward.

Text index i	T[i]	q before	Mismatch fallback	q after
0	a	0	None	1
1	a	1	None	2
2	a	2	None	3
3	a	3	None	4
4	a	4	None	5
5	a	5	P[5]=b mismatches; q=π[4]=4; match the same a against P[4]	5
6	a	5	5→4, then match P[4]	5
7	a	5	5→4, then match P[4]	5
8	a	5	5→4, then match P[4]	5
9	b	5	Matches P[5]=b; no fallback	6

Chapter 2 worked solutions (continued)

At $i = 9$, $q = 6 = |P|$, so the first match starts at $9 - 6 + 1 = 4$. Naive matching compares five a symbols again at each start 0, 1, 2, and 3, then mismatches where it expects b. Start 4 takes six comparisons to succeed. The total is $4 \cdot 6 + 6 = 30$ symbol comparisons. On longer texts of this form, repeated work can reach $\Theta(NM)$. Every KMP fallback decreases q, and forward extensions pay for all increases, giving $O(N + M)$ time including prefix construction.

DS02-04 • Does CSR always save space?

COO stores two indices and one value per entry, giving `16k` bytes. CSR stores a column and value per entry plus row pointers, giving `12k+4(R+1)` bytes. CSR is smaller exactly when `R+1<k`. For the given values COO uses 1,600 bytes and CSR 4,001,204 bytes. Row pointers dominate. CSR may still help row operations, but it does not always save space. Headers, alignment, compression, and alternate index widths are excluded, so these formulas are not universal exact allocation sizes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 2 worked solutions (continued)

Verification code appendix: transpose.cpp

```
#include <algorithm>
#include <cstddef>
#include <iostream>
#include <stdexcept>
#include <tuple>
#include <vector>

struct Entry {
        std::size_t row;
        std::size_t col;
        int value;
        bool operator==(const Entry& other) const {
                return std::tie(row, col, value) == std::tie(other.row, other.col,
other.value);
        }
};

struct Sparse {
        std::size_t rows;
        std::size_t cols;
        std::vector<Entry> entries;
};

void validate(const Sparse& matrix) {
        for (std::size_t i = 0; i < matrix.entries.size(); ++i) {
                const auto& item = matrix.entries[i];
                if (item.row >= matrix.rows || item.col >= matrix.cols || item.value == 0) {
                        throw std::invalid_argument("invalid sparse entry");
                }
                if (i != 0) {
                        const auto& before = matrix.entries[i - 1];
                        if (std::tie(before.row, before.col) >= std::tie(item.row, item.col))
{
                                throw std::invalid_argument("entries must be sorted and
unique");
                        }
                }
        }
}

Sparse transpose(const Sparse& matrix) {
        validate(matrix);
        std::vector<std::size_t> count(matrix.cols, 0);
        for (const auto& item : matrix.entries) {
```

Chapter 2 worked solutions (continued)

```
                ++count[item.col];
        }
        std::vector<std::size_t> next(matrix.cols, 0);
        for (std::size_t col = 1; col < matrix.cols; ++col) {
                next[col] = next[col - 1] + count[col - 1];
        }
        Sparse result{matrix.cols, matrix.rows, std::vector<Entry>(matrix.entries.size())};
        for (const auto& item : matrix.entries) {
                result.entries[next[item.col]++] = {item.col, item.row, item.value};
        }
        return result;
}

void require(bool condition, const char* message) {
        if (!condition) {
                throw std::runtime_error(message);
        }
}

int main() {
        const Sparse matrix{3, 4, {{0, 1, 5}, {1, 0, -2}, {1, 3, 7}, {2, 1, 4}}};
        const auto result = transpose(matrix);
        const std::vector<Entry> expected{{0, 1, -2}, {1, 0, 5}, {1, 2, 4}, {3, 1, 7}};
        require(result.rows == 4 && result.cols == 3 && result.entries == expected,
                "transpose does not match slide example");
        const auto restored = transpose(result);
        require(restored.rows == matrix.rows && restored.cols == matrix.cols &&
                restored.entries == matrix.entries, "double transpose changed matrix");
        const auto empty = transpose(Sparse{0, 4, {}});
        require(empty.rows == 4 && empty.cols == 0 && empty.entries.empty(), "empty
dimensions lost");
        bool rejected = false;
        try {
                transpose(Sparse{2, 2, {{1, 0, 7}, {0, 0, 2}}});
        } catch (const std::invalid_argument&) {
                rejected = true;
        }
        require(rejected, "unsorted input was accepted");
        std::cout << "PASS transpose: exact COO, double transpose, empty, invalid input\n";
}
```

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study